

Rapport de projet final

Optical Character Recognition

Destructeur 2 Mots



Kristen Couty - Romain Dischamp - Liam Chevassus
Timothée Gallier

D2

14 Décembre 2025

Tables des matières

1	Présentation du projet	2
1.1	Nature du projet	2
1.2	Membres	3
1.2.1	Kristen Couty	3
1.2.2	Romain Dischamp	3
1.2.3	Liam Chevassus	3
1.2.4	Timothée Gallier	4
2	Bilan des réalisations	5
2.1	Traitement de l'image	5
2.1.1	Bilan des réalisations	5
2.1.2	Défis rencontrés	12
2.2	Détection de la grille/lettres/mots	14
2.2.1	Bilan des réalisations	14
2.2.2	Défis rencontrés	18
2.3	Réseau de neurones	20
2.3.1	Bilan des réalisations	20
2.3.2	Défis rencontrés	23
2.4	Algorithme de résolution	25
2.4.1	Bilan des réalisations	25
2.4.2	Défis rencontrés	26
2.5	Résolution de la grille	28
2.5.1	Reconstruction de la grille	28
2.5.2	Reconstruction de la liste de mots	29
2.5.3	Résolution de la grille	30
2.5.4	Défis rencontrés	31
2.6	Interface graphique	32
2.6.1	Bilan des réalisations	32
2.6.2	Défis rencontrés	38
3	Gestion de projet	39
3.1	Répartition et avancement du travail	39
4	Conclusion	40
5	Annexes	41
5.0.1	Termes techniques et définitions	41
5.0.2	Bibliographie	43

1 Présentation du projet

1.1 Nature du projet

Ce projet vise à développer un logiciel innovant de reconnaissance optique de caractères (*OCR*) capable de résoudre automatiquement des grilles de mots cachés. Les grilles de mots cachés, très populaires comme jeu de réflexion, consistent en un tableau de lettres dans lequel il faut identifier des mots dissimulés horizontalement, verticalement ou en diagonale. L'objectif de notre application est de prendre en entrée une image représentant une telle grille et d'en produire une version entièrement résolue.

Dans sa version finale, le logiciel offrira une interface graphique intuitive permettant à l'utilisateur de charger des images dans des formats standards, de visualiser la grille, et d'appliquer des corrections sur l'image si nécessaire pour améliorer la qualité de reconnaissance. Une fois traitée, la grille pourra être affichée sous sa forme complète et résolue, avec la possibilité de sauvegarder le résultat pour une utilisation ultérieure.

Un aspect essentiel de ce projet réside dans l'intégration d'un module d'apprentissage automatique. Ce module permettra de créer et d'entraîner un réseau de neurones capable de reconnaître les caractères présents dans les grilles, avec la possibilité de sauvegarder et de recharger les modèles d'apprentissage afin d'améliorer les performances du logiciel au fil du temps.

Ainsi, ce projet combine à la fois des compétences en traitement d'images, en intelligence artificielle et en conception d'interfaces utilisateur, offrant une solution complète et interactive pour résoudre automatiquement des grilles de mots cachés à partir d'images.

1.2 Membres

1.2.1 Kristen Couty

Dans ce projet, je suis en charge de la conception et du développement du réseau de neurones qui est au cœur de la partie d'apprentissage automatique de l'application. Ma mission consiste à créer un modèle capable de reconnaître et d'identifier avec précision les caractères présents dans les grilles de mots cachés, puis de l'entraîner sur des ensembles de données adaptés afin d'améliorer ses performances au fil du temps.

En parallèle, je m'occupe également de la coordination du projet, veillant à la cohérence entre les différentes parties du logiciel et au respect des objectifs fixés, tout en facilitant la collaboration entre les membres de l'équipe. Mon rôle combine donc à la fois un aspect technique, lié à l'intelligence artificielle, et un aspect organisationnel, garantissant la progression fluide et structurée du projet.

1.2.2 Romain Dischamp

Mes missions principales dans ce projet sont les suivantes : Tout d'abord, je dois concevoir un algorithme capable de prendre en entrée un tableau de caractères représentant la grille de mots cachés, ainsi qu'un mot à rechercher. L'objectif de cet algorithme est de déterminer si le mot est présent dans la grille et de renvoyer les coordonnées de début et de fin du mot trouvé. Cet algorithme constitue la dernière brique de la grande pipeline du projet, et joue donc un rôle essentiel dans le bon fonctionnement de l'ensemble.

Je suis aussi chargé du développement d'un programme de détection et d'extraction de caractères présents dans les images du dataset. Le but étant d'extraire toutes les lettres, pour que le futur OCR qui sera réalisé par Kristen analyse chaque image de lettre, et renvoie le caractère associé.

1.2.3 Liam Chevassus

Dans ce projet, je suis responsable du traitement de l'image et de l'interface graphique. Ainsi, je suis en charge du chargement et du traitement de l'image, ce qui regroupe diverses opérations effectuées sur une image : chargement de l'image dans l'interface graphique, conversion en noir et blanc et rotation de l'image. De plus, mon travail est également d'améliorer la qualité de l'image, en supprimant les grains, par exemple.

Par conséquent, mon rôle est de faire le prétraitement de l'image, afin que l'image puisse être envoyée au réseau de neurones pour que la grille soit résolue. L'image doit donc être de la plus haute qualité possible. Enfin, je contribue également au développement de l'interface graphique, qui permet à l'utilisateur de voir le résultat final avec la grille résolue, mais aussi d'agir sur certains paramètres. Je m'attache donc à ce que l'expérience utilisateur soit la plus agréable possible, en produisant une solution combinant ergonomie de l'interface graphique et efficacité du processus de résolution de la grille.

1.2.4 Timothée Gallier

Dans ce projet, je suis en charge d'une partie du traitement de l'image, sur tout ce qui est de la découpe de l'image. Cette découpe est très importante, car elle permettra de donner les bonnes images et les bons caractères au réseau de neurones afin que celui-ci s'entraîne correctement et ne fasse pas d'erreurs liées à une mauvaise qualité de l'image. Je suis également en charge de la résolution et de la reconstruction des différentes grilles. Mon rôle est donc principalement technique, sur la gestion des librairies *GTK* et *GDK* afin de donner le meilleur au réseau de neurones pour que celui-ci soit efficace lors de la réalisation de sa tâche.

Dans la deuxième partie de ce projet mon rôle a été d'améliorer l'interface graphique préexistante et de permettre d'utiliser les différentes fonctions réalisées et implémentées. J'ai donc dû gérer l'interface et chercher comment rendre celle-ci complète mais aussi facilement abordable pour l'utilisateur. De plus, j'ai réalisé une grande partie de la fonction de rotation automatique, implémentant les bases de la détection d'angle avec les fonctions de calcul de variances sur l'image.

2 Bilan des réalisations

2.1 Traitement de l'image

2.1.1 Bilan des réalisations

Avant tout traitement de l'image, il faut d'abord charger cette image et y effectuer un prétraitement. Ce prétraitement inclut plusieurs opérations fondamentales permettant la lisibilité de l'image dans le processus de détection des lettres :

- Conversion de l'image en niveau de gris.
- Détermination du seuil optimal pour la conversion en noir et blanc avec la méthode du seuil d'Otsu.
- Conversion de l'image en noir et blanc en binarisant l'image.
- Détection de l'angle optimal de rotation de l'image (rotation automatique).
- Rotation de l'image avec l'angle optimal de rotation obtenu.
- Réduction du bruit de l'image avec un filtre médian.

Conversion en niveaux de gris

La conversion en niveaux de gris est la première étape du traitement de l'image et permet ensuite de réaliser la conversion en noir et blanc. Pour ce faire, on parcourt chaque pixel du *GdkPixbuf* (pointeur contenant la dimension de l'image, et les valeurs RGB de chaque pixel), et on applique une formule pour convertir les 3 valeurs RGB de chaque pixel en une seule valeur appelée **niveau de gris**. La formule suivante est la formule de luminance qui permet une conversion en niveau de gris par pondération perceptuelle :

$$Gris = 0,299 \times R + 0,587 \times G + 0,114 \times B$$

Les coefficients sont différents entre les valeurs R, G et B car l'œil humain ne perçoit pas les 3 couleurs de la même manière. En effet, il est beaucoup plus sensible au vert qu'au rouge, et encore moins sensible au bleu.

	P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
	E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
	A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
IMAGINE	S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
RELAX	Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
COOL	J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
RESTING	H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
BREATHE	H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
EASY	R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
TENSION	E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
STRESS	C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
CALM	L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
	W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
	B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
	M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
	T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
	Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

Figure 1: L'image 1 du niveau 1 convertie en niveaux de gris

Conversion en noir et blanc

Une fois l'image convertie en niveaux de gris, on peut maintenant la convertir en noir et blanc, en appliquant un processus appelé binarisation de l'image. Celui-ci utilise un seuil. Il est généralement fixé au milieu (128), mais peut aussi être calculé par des méthodes plus efficaces. Chaque pixel en dessous de ce seuil (en niveaux de gris) devient noir (R, G et B sont à 0), sinon il devient blanc (R, G et B sont à 255).

Nous avons pour cela utilisé un seuil arbitraire à 128. Cependant, cela convertissait mal certaines parties de grille, car les images ont toutes un contraste plus ou moins élevé, ce qui fait que chaque image nécessite un seuil spécifique et non un seuil arbitraire commun à toutes les images.

Nous avons donc ensuite opté pour la méthode du seuil moyen, qui calcule le seuil moyen de l'image en faisant la somme de tous ses niveaux de gris et en divisant par le nombre de pixels de l'image. Là encore, cette méthode est plus efficace mais rendait certaines parties de l'image illisibles (par exemple, toute la liste de mots devient blanche).

C'est pourquoi nous avons opté pour une dernière méthode plus efficace, appelée *Seuil d'Otsu*. Cette méthode consiste à séparer l'image en 2 parties : le premier plan et l'arrière-plan. Pour cela, on calcule d'abord un histogramme de luminance allant de 0 à 255, et pour chaque pixel, on rajoute 1 à l'index de sa valeur de niveau de gris. Ainsi, on obtient finalement le nombre de pixels pour chaque valeur de niveau de gris entre 0 et 255.

Une fois l'histogramme obtenu, on cherche le seuil optimal qui maximise la variance inter-classe, c'est-à-dire la variance entre les pixels sombres (noirs) et les pixels clairs (blancs). On calcule donc cette variance pour chaque seuil allant de 0 à 255, et le seuil qui maximise cette variance est le seuil obtenu.

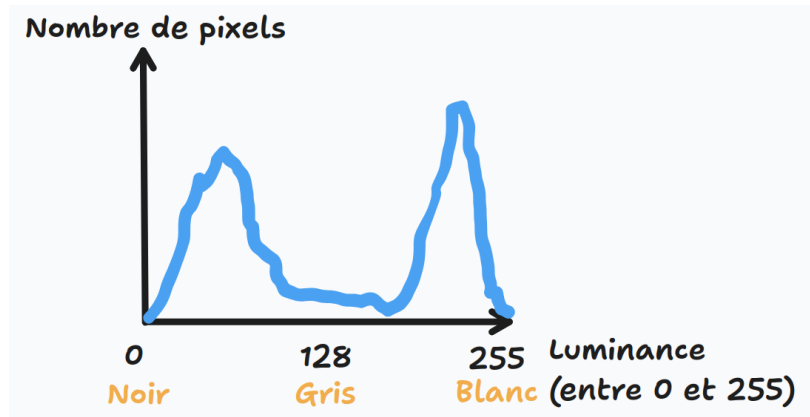


Figure 2: Représentation d'un histogramme pour le seuil d'Otsu

Nous pouvons donc maintenant appliquer la binarisation sur l'image en niveau de gris avec ce seuil, et non un seuil arbitraire. Cela permet que chaque grille soit bien en noir et blanc avec un seuil adapté, et donc que tous les pixels de la liste de mots mais aussi de la grille deviennent noirs. On parcourt donc tous les pixels de la grille, et pour chaque valeur de gris, on détermine si le pixel devient noir ou blanc :

- Si la valeur de niveau de gris est inférieure au seuil, le pixel devient noir (RGB à 0).
- Sinon, le pixel devient blanc (RGB à 1).

	P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
	E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
	A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
IMAGINE	S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
RELAX	Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
COOL	J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
RESTING	H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
BREATHE	H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
EASY	R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
TENSION	E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
STRESS	C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
CALM	L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
	W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
	B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
	M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
	T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
	Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

Figure 3: L'image 1 du niveau 1 convertie en noir et blanc

Détection de l'angle optimal de rotation

De la même manière que pour les conversions expliquées ci-dessus, la rotation de l'image consiste à faire tourner l'image d'un certain angle et à effectuer une rotation sur chaque pixel de l'image d'origine de cet angle, pour donner une image d'arrivée qui a subi cette rotation.

Tout d'abord, on calcule l'angle de rotation optimal pour l'image, afin que l'image soit parfaitement droite une fois la rotation effectuée. Il nous faut donc un score dépendant de l'orientation de l'image, qui, plus l'image est droite, est élevé, nous indiquant l'angle pour lequel l'image est parfaitement droite.

Le score que nous avons utilisé s'appelle la **projection horizontale**. Ce score est calculé dans la fonction `compute_projection_variance`. Cette fonction mesure à quel point les lignes horizontales sont bien marquées. Pour chaque ligne, on calcule la différence entre 2 pixels adjacents, puis on calcule la variance de cette projection.

$$\text{somme de la ligne} += |\text{pixel}(x) - \text{pixel}(x - 1)|$$

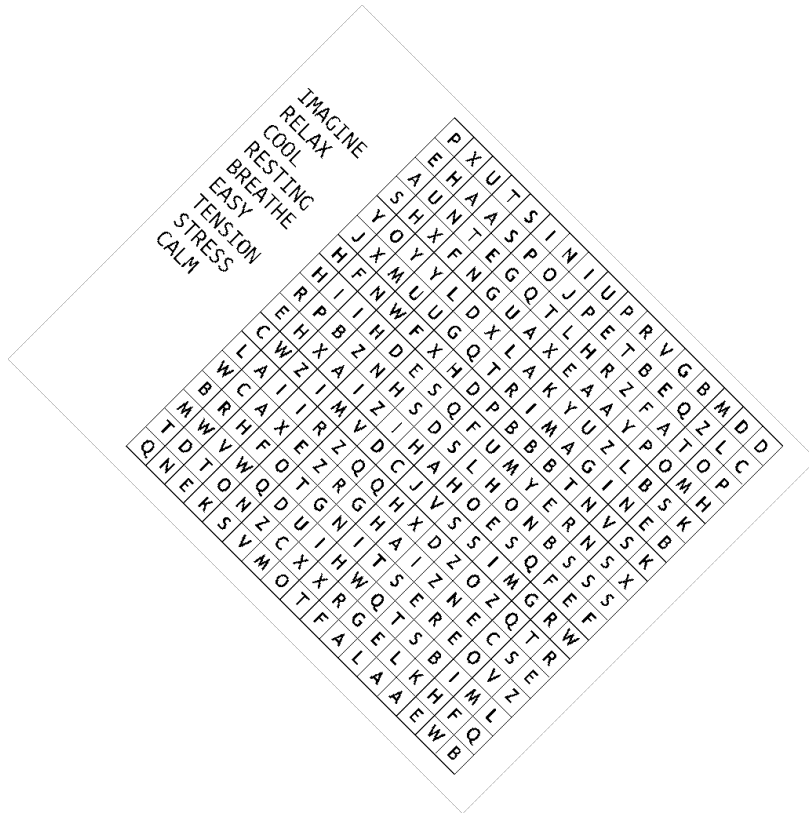


Figure 4: L'image 1 du niveau 1 ayant subi une rotation d'angle 45° vers la droite

Il y a donc deux scénarios en fonction de si l'image est droite ou non:

- Soit l'image est droite et dans ce cas on aura une variance plus élevée car les lignes formées seront sur une seule rangée de pixels.

- Soit l'image n'est pas droite et dans ce cas les lignes formées dans l'image seront réparties sur plusieurs rangées de pixel, faisant ainsi baisser le score de la variance.

Rotation automatique de l'image

La réalisation de la rotation de l'image étant déjà effectuée, il est maintenant important de faire la rotation automatique car on ne veut pas avoir à donner l'angle de l'image à la main. Il existe donc une fonction qui permet de détecter le meilleur angle de rotation afin de donner cet angle à la fonction déjà existante, `detect_best_angle`.

La fonction a besoin d'un prétraitement de l'image, à savoir une image binarisée en noir et blanc. Elle va ensuite réaliser une diminution de l'échelle de l'image afin d'obtenir une image plus petite, permettant d'éviter de calculer des millions de pixels pour trouver le meilleur angle. La fonction fonctionne en plusieurs étapes :

- Réduction de la taille de l'image (Etape 1)
- Recherche grossière de l'angle optimal par 5° (Etape 2)
- Recherche fine autour l'angle trouvé pour trouver le meilleur angle à 0.1° près (Etape 3)
- Appel à la fonction `rotate_image(pixbuf, angle)` une fois l'angle trouvé (Etape 4)

Etape 1: Ici nous allons réduire proportionnellement l'échelle de l'image avec la fonction `downscale_pixbuf` en lui donnant en paramètre la taille maximale de l'image compressée souhaitée, ici nous lui donnerons `150 * 150 pixels` ce qui est largement suffisant pour appliquer nos algorithmes de tests avec une meilleure optimisation de temps de calcul.

Etape 2: L'idée finale de la fonction est de trouver le meilleur angle à 0.1° près, néanmoins si il faut faire tous les tests à 0.1° dans l'intervalle $[-90^\circ ; 90^\circ]$ cela représenterait `180 * 10 = 1800 tests` de `150*150 pixels`, ce qui prendrait une éternité à calculer. C'est pour cela que l'algorithme va d'abord trouver l'angle général en testant tous les 5° dans l'intervalle $[-45^\circ ; 45^\circ]$, ce qui réduit les tests au nombre de `90 / 5 = 18 tests`

Etape 3: Une fois l'angle général trouvé la fonction va ensuite relancer le test sur l'intervalle $[\text{angle} - 3^\circ ; \text{angle} + 3^\circ]$, réalisant ainsi `6 * 10 = 60` nouveaux tests afin de trouver l'angle optimal.

Etape 4: Enfin, la dernière étape de cette fonction est l'appel à la fonction `rotate` classique avec l'angle trouvé précédemment.

Conclusion: En résumé, la fonction va d'abord faire des tests généraux sur l'image puis ensuite relancer des calculs sur un intervalle plus fermé, permettant ainsi à la fonction de s'exécuter quasiment instantanément. Au final, il y aura 2 options :

- Soit l'image est droite et dans ce cas on aura une variance plus élevée car les lignes formées seront sur une seule rangée de pixels.
- Soit l'image n'est pas droite et dans ce cas les lignes formées dans l'image seront réparties sur plusieurs rangées de pixel, faisant ainsi baisser le score de la variance.

Finalement, les images droites restent droites (angle de rotation à 0°), et les autres images subissent une rotation afin qu'elles deviennent droites. On obtient donc une rotation automatique précise à $0,1^\circ$ près qui permet, par la suite, à l'OCR de travailler sur une image droite.

```

A A S S E M B L Y O O Y H O
H D I B O V P D S H T Y H B
J L L M M T D Y L E O P R L   H E L L O
L B O R S A C O P L Y C U A   W O R L D
V A A M O J O I P Y O Y S Y   R U S T
T S M A I W S M S T L R T A   P H P
P L M L H Y S J S A L O A V   J A V A
P H P H I P T R U W B T H A   A S S E M B L Y
R W M R H T M L T J P Y S J   P Y T H O N
H A T L T I D E S L H O M A   H T M L
O U S L L B J A O J O Y O B   M O J O
O O S O S H M L M H S L T T   B A S I C
S R T A P Y T H O N Y A A L
P N N H E L L O H M O O L V

```

Figure 5: Traitement complet avec rotation automatique avec meilleur angle de rotation à $4,6^\circ$ pour l'image 2 du niveau 2

Découpe des images

Afin que l'*OCR* final puisse reconnaître les caractères et les résoudre, il est essentiel de lui donner des images bien formatées, ne contenant qu'un caractère à la fois, afin de permettre au réseau de neurones d'apprendre et d'analyser. Il a donc fallu faire des fonctions prenant en charge une image et l'adaptant à différents besoins.

La première fonction implémentée a été une découpe de l'image en deux parties, de manière verticale ou horizontale, qui sera utile afin de séparer la partie liste des mots à trouver et la partie contenant la grille de lettres. Les images sont gérées par l'interface *GTK3* qui utilise elle-même une sous-partie appelée *GDK* avec des *GdkPixBuf*, là où sont stockées les images que l'on peut ensuite manipuler et afficher de la manière que nous souhaitons.

La deuxième fonction réalisée sera utilisée pour découper des images carrées en parts égales, carrées elles aussi, avec le nombre de découpes souhaitées. La 3ème et dernière fonction pour le moment sert quant à elle à découper un rectangle aux coordonnées souhaitées de (x1, y1) à (x2, y2). Cette fonction sera utilisée notamment pour découper les lettres et extraire les caractères un à un, notamment lors de la détection de la grille ainsi que les lettres et les mots.



A	U	N
S	H	X
Y	O	Y
J	X	M
H	F	N
H	I	I

Figure 6: Exemple d'utilisation de la fonction crop sur un morceau de la grille de l'image 1 du niveau 1

2.1.2 Défis rencontrés

Traitements

Pour les traitements, nous avons tout d'abord eu du mal à sauvegarder l'image en fichier *PNG*. Nous utilisons la fonction `save_pixbuf_as_png(GdkPixbuf *pixbuf, char *filename)` qui permet de transformer un Pixbuf en image sous format *PNG*. Cependant, nous avons eu des difficultés avec le chemin de l'image, car en fonction du dossier dans lequel la fonction est exécutée, l'image est introuvable. Il a donc fallu écrire une fonction qui construit le chemin absolu de l'image, nommée `get_image_path(char *filename)` pour que l'image puisse être trouvée depuis n'importe où dans le projet. De manière générale, nous avons écrit plusieurs fonctions d'aide pour la construction des chemins des fichiers et du chargement des images que nous utilisons dans beaucoup de fichiers différents.

Rotation

La rotation était, elle aussi, complexe, car non seulement il fallait faire pivoter l'image d'un certain angle pour avoir le pixel d'arrivée, mais il fallait également le faire correspondre avec le pixel de départ, en faisant une rotation inverse. De plus, il fallait trouver la bonne formule avec le cosinus et le sinus de l'angle pour trouver le pixel correspondant à la rotation.

Rotation automatique

Le premier défi rencontré a été de trouver l'idée du calcul des lignes pour vérifier la variance. Le premier prototype réalisait une fonction simple pour la moyenne à l'aide d'une formule mathématique simple :

$$\text{Var}(X) = E[X^2] - (E[X])^2.$$

Plusieurs problèmes se sont alors posés :

- Comment calculer l'image avec les différentes couleurs qui ne valent pas les mêmes valeurs ?
- Comment gérer les bruits parasites qui pourraient dérégler le calcul ?

La principale solution a été de binariser l'image en noir et blanc ainsi que d'appeler une fonction `median_filter` qui permet d'éliminer la plupart des bruits. De plus la fonction de calcul a été améliorée pour prendre en compte un calcul horizontal et vertical, améliorant ainsi la précision de celle-ci.

Axes d'améliorations possibles :

- Arriver à trouver une fonction de calcul plus efficace et moins coûteuse
- Trouver une dimension optimale plus basse que les 150pixels, permettant une réduction du coût de calcul
- Si l'image n'a pas besoin d'être tournée, le savoir dès le début, qui éviterait bon nombres de calculs
- Effectuer plus de tests avec grands angles, afin de réduire le nombre de tests généraux de 18 à une 10aine si possible
- Si possible : Multi thread pour calculer les différents tests en parallèle et gagner du temps de calcul

Découpe des images

La principale difficulté rencontrée au début de ce projet fut la gestion de l'image, à travers les différentes couches de *GTK* et *GDK* et les différents pointeurs. À cela s'ajoutait la complexité de programmer en *C* avec la gestion manuelle de la mémoire, l'allocation nécessaire pour stocker tous les *PixBuf* des différentes images découpées. L'idée de départ pour les nouvelles images ajustées était de les enregistrer sous un format *png*, mais nous nous sommes rendu compte que cela était bien trop coûteux en mémoire, car cela impliquait d'enregistrer une première fois l'image en *png*, puis la remettre en *PixBuf* pour enfin la remettre à nouveau en *png*, tout en ayant en tête que chercher et charger un fichier est assez coûteux. La solution a donc été d'allouer un tableau de pointeurs de *PixBuf* qui seront ensuite réutilisés dans d'autres fonctions.

2.2 Détection de la grille/lettres/mots

Ce projet a pour but final de résoudre une grille de mots cachés en trouvant les coordonnées de chacun des mots dedans. C'est pourquoi il est obligatoire d'avoir, à la fin de la chaîne de traitement du projet, un algorithme prenant en entrée un tableau de caractères ainsi qu'une liste de mots, et d'avoir en sortie les coordonnées de chacun des mots dans la grille. De plus, à cet algorithme, on vient ajouter plusieurs fonctions permettant de recréer les grilles des images (grâce aux sorties de la détection de grille, mots et lettres) et permettant, quand la grille est résolue, de recalculer les coordonnées des mots dans l'image.

2.2.1 Bilan des réalisations

1ère soutenance

Pour ce faire, nous avons utilisé différentes fonctions créées par les membres du projet, dont ceux responsables du traitement de l'image. Voici la chaîne de traitement du programme permettant de détecter les lettres.

Premièrement, nous initialisons deux *Pixbufs*, tous deux issus de l'image en entrée. Le premier nous permettra de trouver les coordonnées de chaque lettre en effectuant plusieurs transformations dessus, et nous couperons le deuxième aux coordonnées retournées par le premier. Les transformations effectuées sur le premier sont les suivantes : nous convertissons l'image en nuances de gris et ensuite nous binarisons l'image. Ensuite, nous parcourons horizontalement et verticalement le tableau pour retirer, s'il y en a, les lignes de la grille.

Pour détecter les lettres, nous parcourons l'image pixel par pixel, jusqu'à trouver un pixel noir. Quand nous en trouvons un, nous parcourons tous ses pixels noirs voisins jusqu'à ne plus trouver de pixels noirs. Pour garder en mémoire les pixels que j'ai visités ou non, nous avons créé un tableau de la même taille que le nombre de pixels de l'image, qui garde un entier à la valeur soit zéro, soit un, et qui agit comme un booléen.

Avant de faire le parcours, nous avons fait une fonction pour détecter si une image est tournée, mais qui pour l'instant renvoie un angle de zéro degré. Cela nous permettra de plus facilement implémenter cette fonction dans le futur.

Maintenant que nous avons les coordonnées de chaque lettre, nous pouvons trouver les coordonnées de la grille ainsi que de la liste de mots. Pour ce faire, nous parcourons chaque lettre de la liste de lettres trouvées, et si une lettre se trouve dans l'intervalle $[x_{\min} - \text{seuil}; x_{\max} + \text{seuil}]$ et $[y_{\min} - \text{seuil}; y_{\max} + \text{seuil}]$ d'un des deux ensembles, alors nous en déduisons qu'elle appartient à cet ensemble. (Nous partons du principe que la liste de mots est à une distance raisonnable de la grille de lettres).

Enfin, comme nous obtenons les lettres appartenant à la liste de mots, nous pouvons parcourir ces lettres et établir des mini-ensembles représentant chaque mot de

la liste, et pour savoir quelle lettre appartient à quel mot, nous procédons de la même manière que précédemment, mais uniquement sur l'axe x.

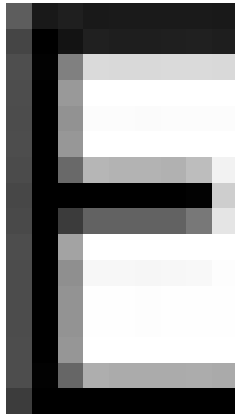
P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

(a) La grille découpée

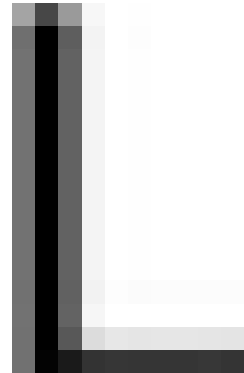
IMAGINE
RELAX
COOL
RESTING
BREATHE
EASY
TENSION
STRESS
CALM

(b) La liste de mots découpée

Figure 7: Quelques images découpées de l'image 1 du niveau 1: La grille et la liste de mots



(a) Une lettre "E" coupée depuis la grille de l'image 1 du niveau 1



(b) Une lettre "L" coupée depuis la liste de mots de l'image 1 du niveau 1

Figure 8: Quelques lettres découpées dans la grille

2ème soutenance

Pour la 2ème soutenance, nous avons principalement rendu les seuils de la pipeline adaptatifs. En effet, nous avons précédemment créé des seuils arbitraires dans la pipeline, notamment lors de la séparation de la grille dans 2 boîtes séparées, la grille et la liste de mots.

De plus, afin de préparer l'implémentation de la résolution de la grille, il fallait aussi sauvegarder les résultats de la pipeline dans une structure permettant de donner au programme de résolution de la grille toutes les informations nécessaires sur la pipeline. C'est pourquoi nous avons créé 3 structures :

Premièrement, nous avons implémenté une structure **Grid** contenant toutes les informations sur la grille:

- **nb_rows** : Le nombre de lignes de la grille.
- **nb_cols** : Le nombre de colonnes de la grille.
- **grid** : Le tableau de caractères à 2 dimensions, contenant **nb_rows** lignes et **nb_cols** colonnes.

Ensuite, nous avons également créé une autre structure **Words** pour sauvegarder les informations concernant la liste de mots de la grille:

- **detected_words_count** : Le nombre de mots détectés après la reconstruction de la liste de mots.
- **words** : Le tableau de caractères à 2 dimensions, contenant **detected_words_count** mots sous forme de chaîne de caractères.
- **solved_words_grid_coos** : Le tableau de coordonnées des mots résolus sur la grille.
- **solved_words_image_coos** : Le tableau de coordonnées des mots résolus sur l'image.

Enfin, nous avons implémenté la structure **PipelineResult** qui, en plus de récupérer les structures **Grid** et **Words**, récupère aussi le nombre de lettres détectées et sauvegarde les données suivantes:

- **rotation_angle** : L'angle de rotation appliqué lors de la rotation automatique.
- **grid_coo** : Les coordonnées des 4 côtés de la grille.
- **words_coo** : Les coordonnées des 4 côtés de la liste de mots.
- **nb_letters** : Le nombre de lettres total détectées.
- **nb_letters_grid** : Le nombre de lettres dans la grille.
- **nb_letters_words** : Le nombre de lettres dans la liste de mots.
- **grid** : La structure **Grid** contenant les informations sur la grille.
- **words** : La structure **Words** contenant les informations sur la liste de mots.

Avec toutes ces structures, la pipeline peut envoyer toutes les coordonnées des lettres de la grille et de la liste de mots au programme de résolution de la grille.

```

[INFO] Best rotation angle : 0,00°
[INFO][SOLVER] Built grid with 17 rows and 17 columns
[INFO][SOLVER] Built words list with 9 words detected
[INFO] Detected grid coordinates : (195, 30)(766, 603)
[INFO] Detected words list coordinates : (24, 123)(133, 398)
[INFO] Number of letters detected : 340 (In the grid : 289, in the words list : 51)
[INFO] Number of words detected in the words list of the grid : 9

```

Figure 9: Exemple de résultats de pipeline pour l'image 1 du niveau 1

```

[INFO] Mots détectés dans la liste de mots : 10
Mot 1 : TROPIC
Mot 2 : BEACH
Mot 3 : SUMMER
Mot 4 : HOLIDAY
Mot 5 : SAND
Mot 6 : BALL
Mot 7 : TAN
Mot 8 : RELAX
Mot 9 : SUN
Mot 10 : FUN
[INFO] Grille détectée : 7 lignes, 8 colonnes
[S U M M E R L H]
[C I P O R T N O]
[B S U N B A L L]
[R E L A X E P I]
[T D A O S A N D]
[A Y B C A Z I A]
[N F U N H R S Y]
[INFO] Best rotation angle : -24,90°
[INFO][SOLVER] Built grid with 7 rows and 8 columns
[INFO][SOLVER] Built words list with 10 words detected
[INFO] Detected grid coordinates : (260, 375)(860, 905)
[INFO] Detected words list coordinates : (975, 360)(1117, 803)
[INFO] Number of letters detected : 102 (In the grid : 56, in the words list : 46)
[INFO] Number of words detected in the words list of the grid : 10

```

Figure 10: Exemple de résultats de pipeline pour l'image 1 du niveau 2

La pipeline est donc synchronisée avec le solver et permet le bon déroulement de la résolution de la grille.

2.2.2 Défis rencontrés

1ère soutenance

Normalement, nous appliquions aussi un *filtre de Sobel* sur l'image pour détecter les contours de chaque élément, cela nous permettait de mieux les découper par la suite, mais après plusieurs tests, nous nous sommes rendu compte que ce n'était pas utile et que sans ce filtre la découpe se faisait assez bien. Donc le développement d'une fonction pour appliquer un *filtre de Sobel* sur une image nous a prit du temps, mais cela nous a permis de nous familiariser avec les *Pixbufs*.

Un autre problème qui apparaîait est que les images du niveau deux et trois contiennent des dessins et/ou sont tournées sur un côté, cela entraîne un dysfonctionnement dans notre algorithme pour trouver les mots ainsi que la grille, car dans le cas d'une rotation de l'image, toutes les lettres d'une même ligne n'ont pas les mêmes coordonnées en ordonnée, et pour les images contenant des dessins, ceux-ci sont détectés comme des lettres et donc rendent les coordonnées de la grille et/ou de la liste de mots invalides. C'est pourquoi il faudrait appliquer certains filtres sur l'image de façon plus radicale pour ne garder que les lettres et mieux épurer l'image des groupements de pixels ne faisant pas référence à des lettres.

2ème soutenance

Après la première soutenance, nous nous sommes rendus compte que certains mots étaient mal détectés par le solver dans la grille. Cette découverte est arrivée avec la batterie de tests ajoutée pour chaque partie du projet. Tous ces tests ont permis de déboguer le problème plus facilement.

Le problème venait de la façon dont on traitait nos images, en effet, certaines lettres étaient bien détectées, d'autres non, et lorsque nous changions les seuils, cela déplaçait juste le problème. Nous essayions de trouver un équilibre entre trop traiter l'image ou pas assez, sauf que l'équilibre parfait n'existait pas. C'est pourquoi nous avons décidé de rendre notre code le plus adaptatif possible. Cela a commencé par la suppression de certains seuils prédéfinis. De plus nous avons implémenté une fonction pour trier les lettres entre elles car parfois les coordonnées de la grille ou des mots était erronée. Enfin, pour détecter si 2 lettres appartiennent au même ensemble, avant nous avions un seuil prédéfini, nous avons fait le choix de calculer ce seuil en fonction de la largeur moyenne des lettres d'un ensemble. Mais malgré tout ces changements, trop de lettre étaient découpées dans l'image 1 niveau 2, cela venait de certaines fonctions de traitements qui créaient des trous dans les lignes de la grille, les transformants en petits blocs, indifférenciables des lettres. Nous avons ainsi choisi de supprimer de la pipeline ces fonctions.

Axes d'améliorations possibles

La détection de lettres, grilles et mots est opérationnelle. Néanmoins plusieurs axes pourraient être à revoir pour rendre le projet plus professionnel et qualitatif.

Premièrement, le parcours dans l'image, pixels par pixels, pour trouver les coordonnées des lettres n'est pas optimal. En effet, lorsqu'on détecte une "lettre" qui s'avère être plus grande que les dimensions normales d'une lettre, on la filtre sans l'ajouter à la liste des lettres. Mais, nous pourrions imaginer d'arrêter le parcours de cette lettre dès qu'une coordonnée n'est pas normale. Cela permettrait d'économiser du temps de recherche en ne parcourant pas, par exemple, les lignes de la grille de mots.

Pour ce qui est de l'optimisation temporelle, nous pourrions aussi réfléchir à une autre option plus efficace pour trouver les lettres. En effet, nous parcourons tout les pixels d'une image pour s'assurer de trouver toutes les lettres, or nous aurions pu utiliser le filtre de Sobel que j'avais développé, combiné à d'autres fonctions et transformations pour effectuer une vraie line-detection, et trouver dans l'image l'emplacement de la grille pour ensuite trouver les lettres dans cette grille. Cela aurait pu réduire en partie le temps que met cette partie du projet, pour trouver les lettres. De plus, cette méthode aurait potentiellement permis de résoudre plus de types de grilles différentes comme par exemple les grilles de niveaux 3.

Enfin, un autre axe d'amélioration possible est l'amélioration de l'adaptabilité de l'algorithme. En effet nous avons surtout testé notre code avec les assets fournis et nous n'avons pas beaucoup diversifié les différents types d'images. Nous aurions pu prendre des images avec une très haute résolution ou une très basse, des images totalement retournées, ou encore inversées en miroir. Cela aurait sûrement permis de mettre le doigt sur différents bugs non révélés avec les images par défaut.

2.3 Réseau de neurones

2.3.1 Bilan des réalisations

Proof of concept

Dans le cadre de ce projet, nous avons implémenté dans un premier temps un *POC* de notre réseau de neurones artificiels en langage C, capable d'apprendre différentes fonctions logiques binaires, telles que *XOR*, *AND*, *OR* et *XNOR*. L'objectif était de mettre en pratique les concepts fondamentaux des réseaux de neurones, notamment le *perceptron*, la *fonction d'activation*, la *rétropropagation* et l'optimisation des poids par *descente de gradient*.

Pour chaque fonction logique, le réseau comporte deux neurones en entrée correspondant aux variables logiques, une couche cachée de deux neurones utilisant la fonction *sigmoïde* afin d'introduire la non-linéarité nécessaire pour apprendre le *XOR* et le *NXOR*, et une couche de sortie composée d'un unique neurone également sigmoïdal. La fonction sigmoïde a été choisie pour sa continuité et sa dérivabilité, indispensables à la *rétropropagation*, et pour sa capacité à produire des valeurs entre 0 et 1 (fonction binaire comparée à *softmax*), correspondant naturellement aux valeurs logiques.

Les poids et les biais ont été initialisés aléatoirement afin de briser toute symétrie et permettre un apprentissage efficace. La *rétropropagation* a permis d'ajuster les poids en fonction de l'erreur entre les sorties prédites et les valeurs cibles. Le taux d'apprentissage a été fixé expérimentalement à 0,5, ce qui a permis une convergence rapide tout en évitant les oscillations. Les tests ont montré que le réseau pouvait correctement reproduire les tables de vérité des fonctions *AND*, *OR* et *XNOR*, et qu'il réussissait également à apprendre le *XOR*, démontrant la capacité du réseau à modéliser des relations non linéaires.

Fonction	Expression	Entrée 00	Entrée 01	Entrée 10	Entrée 11
AND	$A \cdot B$	0	0	0	1
OR	$A + B$	0	1	1	1
XOR	$A \oplus B$	0	1	1	0
XNOR	$\overline{A} \cdot \overline{B} + A \cdot B$	1	0	0	1

Table 1: Tables de vérité des fonctions logiques utilisées pour l'apprentissage du réseau de neurones.

```

--- Tests XNOR (!A.!B + A.B) ---
Epoch 0 - Error: 1.049113
Epoch 1000 - Error: 0.393433
Epoch 2000 - Error: 0.013107
Epoch 3000 - Error: 0.006015
Epoch 4000 - Error: 0.003846
Epoch 5000 - Error: 0.002810
Epoch 6000 - Error: 0.002208
Epoch 7000 - Error: 0.001815
Epoch 8000 - Error: 0.001539
Epoch 9000 - Error: 0.001334
Input: 0 0, Predicted: 1, Expected: 1
Input: 0 1, Predicted: 0, Expected: 0
Input: 1 0, Predicted: 0, Expected: 0
Input: 1 1, Predicted: 1, Expected: 1
All tests passed.

```

Figure 11: Exemple de prédiction du PoC de notre réseau de neurones

Reconnaissance de caractère

Pour permettre à notre programme d'identifier automatiquement les lettres présentes dans une grille, nous avons développé un réseau de neurones capable d'apprendre à reconnaître les formes graphiques correspondant aux caractères de l'alphabet. À partir du réseau initial utilisé pour apprendre des fonctions logiques, nous avons progressivement enrichi l'architecture et ajouté de nouvelles fonctionnalités afin d'obtenir un modèle réellement capable d'effectuer une tâche de reconnaissance visuelle.

Contrairement au *XOR* qui utilisait la fonction *sigmoïde*, notre réseau utilise désormais *ReLU* pour la couche cachée. *ReLU* est une activation moderne, plus efficace pour des réseaux profonds ou contenant de nombreuses unités, car elle ne sature pas et favorise une propagation du gradient plus stable. De son côté, la couche de sortie utilise désormais *Softmax*, indispensable pour les problèmes de classification multi-classes. *Softmax* convertit les valeurs produites par le réseau en probabilités normalisées, rendant l'interprétation immédiate : la lettre prédite correspond simplement à la sortie qui possède la probabilité la plus élevée.

Pour remplacer *l'erreur quadratique* utilisée précédemment par notre PoC, nous avons introduit la *Cross-Entropy Loss*, spécialement conçue pour la classification multi-classes. Elle mesure la différence entre la distribution prédite par le *Softmax* et la distribution cible (*one-hot*). Elle possède plusieurs avantages : un gradient plus informatif, un apprentissage plus rapide et un comportement stable même lorsque le modèle est très sûr de ses prédictions. De plus, grâce à la combinaison *Softmax + CrossEntropy*, la dérivation devient plus simple : l'erreur finale est simplement *output - target*, ce qui permet une implémentation à la fois optimisée et robuste.

Pour passer d'un simple réseau conçu pour apprendre des fonctions logiques à un modèle capable de reconnaître des lettres, il a été nécessaire d'introduire un véritable *dataset* annoté. Nous avons défini une architecture standard inspirée d'*EMNIST* : un dossier contenant 26 sous-dossiers (de A à Z), chacun regroupant les images représentant la lettre correspondante. Une nouvelle fonction `load_dataset` parcourt automatiquement cette structure, détecte chaque image, la charge, la transforme en vecteur numérique et génère en parallèle un vecteur *one-hot* pour représenter la classe cible (26 sorties possibles). Ce dataset est ensuite utilisé pour alimenter l'entraînement par gradient descent. Grâce à cette nouvelle organisation, le réseau apprend à généraliser à partir de nombreux exemples réels plutôt qu'à partir de quelques points codés en dur. Pour construire le dataset, nous nous sommes servi de nos fonctions pour découper la grille afin d'isoler les lettres que nous avons organisées de la manière expliquée précédemment. Cette méthode a suffi pour l'élaboration d'un *dataset* assez performant pour prédire des lettres capitales écrites à l'ordinateur.

Comme les images brutes ne peuvent pas être utilisées directement par le réseau de neurones, plusieurs étapes de prétraitement ont été ajoutées. D'abord, chaque image est donnée à l'*OCR* après avoir été convertie en noir et blanc, ce qui réduit le bruit et supprime des informations non pertinentes comme la couleur. Cela permet aussi d'appliquer plusieurs traitements sur l'image avant de lancer la prédiction (cf. partie 2.1 - Traitement de l'image). Ensuite, toutes les images sont redimensionnées en 28×28 pixels, une taille standard pour du machine learning (*MNIST/EMNIST*), ce qui permet d'avoir une dimension d'entrée constante pour le réseau, ce qui est obligatoire. Les pixels sont ensuite normalisés entre 0 et 1 et inversés (le texte devient clair, le fond sombre), ce qui améliore la stabilité du gradient. Ces transformations automatiques rendent les données cohérentes, homogènes et adaptées à l'apprentissage.

```
--- Testing function predict_letter() with Images ---  
  
[SUCCESS] Image 'A' -> Predicted 'A'  
[SUCCESS] Image 'B' -> Predicted 'B'  
[SUCCESS] Image 'C' -> Predicted 'C'  
[SUCCESS] Image 'I' -> Predicted 'I'  
[SUCCESS] Image 'J' -> Predicted 'J'  
[SUCCESS] Image 'M' -> Predicted 'M'  
[SUCCESS] Image 'Z' -> Predicted 'Z'
```

Figure 12: Exemple de prédiction de caractères

2.3.2 Défis rencontrés

Proof of concept

L'une des principales difficultés de ce projet réside dans la compréhension et la gestion de multiples abstractions mathématiques et informatiques. La conception d'un réseau de neurones implique de manipuler des notions telles que les vecteurs de poids, les fonctions d'activation non linéaires, la rétropropagation et les dérivées partielles pour calculer les gradients. Cette complexité se cumule avec la nécessité de programmer en *C*, où la gestion manuelle de la mémoire et la structuration correcte des matrices et vecteurs de poids sont critiques. Comprendre comment chaque neurone contribue à la sortie finale et comment l'erreur se propage à travers les couches demande une forte capacité d'abstraction et une vision claire du flux des données. L'ajustement des *hyperparamètres*, tels que le taux d'apprentissage et l'initialisation des poids, ajoute un niveau de complexité supplémentaire, car ils influencent directement la convergence et la stabilité de l'apprentissage. Ces aspects ont nécessité plusieurs itérations expérimentales et un suivi attentif pour s'assurer que le réseau converge correctement vers la solution attendue.

Reconnaissance de caractère

Lors du développement de notre réseau de neurones chargé de reconnaître les caractères extraits des images, nous avons été confrontés à un problème majeur : la *fonction de perte* ne diminuait pas au fil de l'entraînement. Même après de nombreuses itérations, le réseau restait bloqué à un niveau d'erreur élevé et ne parvenait pas à apprendre correctement.

Dans une première version, les poids du réseau étaient initialisés de manière complètement aléatoire, sans contrainte sur leur amplitude. Cette stratégie naïve pose deux problèmes principaux :

- Variances incohérentes entre les couches : si les *poids* sont trop grands dès le départ, chaque propagation transforme les valeurs d'entrée en nombres de plus en plus extrêmes. Les *activations* saturent alors (par exemple la *sigmoïde* reste bloquée à 0 ou 1), ce qui annule le *gradient*.
- Explosions numériques: dans certains cas, les valeurs calculées devenaient si élevées que les *poids* “divergeaient” vers l'infini dès les premières itérations, rendant la mise à jour instable et la perte totalement plate.

Ce phénomène est bien connu dans les réseaux de neurones : une mauvaise initialisation peut empêcher tout apprentissage. Nous avons résolu ce problème en adoptant une initialisation adaptée, de type *Xavier* pour les couches utilisant une activation *sigmoïde/tanh*, ou *He* pour les couches utilisant *ReLU*. Ces méthodes contrôlent la variance des *poids* et stabilisent la propagation du *gradient*.

Un autre élément nous a également retardés dans la conception du réseau de neurones : la taille des images du *dataset* ne correspondait pas à la dimension attendue par la couche d'entrée du réseau.

Dans notre première implémentation, nous chargions directement les images extraites du *dataset* sans les redimensionner ni les normaliser. Or, un réseau de neurones nécessite que toutes les entrées aient une dimension strictement identique.

Chaque image doit être convertie en un vecteur d'entrée dont la longueur est exactement égale au nombre de neurones de la couche d'entrée. Sans cela, soit le réseau reçoit un vecteur trop long entraînant un écrasement ou une perte d'informations arbitraire, soit il reçoit un vecteur trop court provoquant une lecture hors limites, valeurs non initialisées. Dans certains cas, le programme continuait l'entraînement avec des données corrompues, produisant des *gradients* incohérents. Le résultat était un apprentissage totalement instable : la perte variait au hasard et les prédictions du réseau étaient chaotiques.

2.4 Algorithme de résolution

Ce projet a pour but final de résoudre une grille de mots cachés en trouvant les coordonnées de chacun des mots dedans. C'est pourquoi il est obligatoire d'avoir, à la fin de la chaîne de traitement du projet, un algorithme prenant en entrée un tableau de caractères ainsi qu'une liste de mots, et d'avoir en sortie les coordonnées de chacun des mots dans la grille.

2.4.1 Bilan des réalisations

Ici, comme nous voulons tester notre *solver* avec des commandes, nous devons prendre en entrée un mot ainsi qu'un fichier texte qui contient le tableau de caractères. Donc dans la fonction `solver`, nous devons transformer le fichier texte en un tableau de caractères pour ensuite effectuer la recherche dans ce tableau. Je commence par effectuer un parcours ligne par ligne jusqu'à tomber sur la même lettre que la première lettre du mot à chercher, et ensuite, à partir de ce point-là, un parcours dans toutes les directions est lancé, s'arrêtant s'il trouve le mot en entier ou si aucune direction n'a donné de résultat satisfaisant. Si le mot n'est pas trouvé, alors l'algorithme continue son parcours jusqu'à trouver le mot ou à arriver à la fin du tableau, ce qui signifie que la recherche est négative.

```
destructeur-2-mots on 1 main [$!] via 2 impure
> cat tests/solver_grid_sample.txt
```

	File: tests/solver_grid_sample.txt
1	HORIZONTAL
2	DXRAHCLBGA
3	DIKCILEOKC
4	IGAJHYLYHI
5	HGFGODTIOT
6	GDLROWKBFR
7	PLNRDNERGE
8	JHAIDUAGJV
9	UKGFFOLLEH

```
destructeur-2-mots on 1 main [$!] via 2 impure
> ./build/solver ./tests/solver_grid_sample.txt horizontal
(0, 0)(9, 0)
```

Figure 13: Démonstration du fonctionnement de l'algorithme de résolution

```

----- Solver Tests -----
--- Horizontal Tests ---
[SUCCESS] Solved word horizontal correctly, got: (0, 0)(9, 0)
[SUCCESS] Solved word HORIZONTAL correctly, got: (0, 0)(9, 0)
[SUCCESS] Solved word LATNOZIROH correctly, got: (9, 0)(0, 0)
[SUCCESS] Solved word hori correctly, got: (0, 0)(3, 0)
[SUCCESS] Solved word HELLO correctly, got: (9, 8)(5, 8)
[SUCCESS] Solved word world correctly, got: (5, 5)(1, 5)
[SUCCESS] Solved word toit correctly, got: (9, 4)(6, 4)
[SUCCESS] Solved word GOD correctly, got: (3, 4)(5, 4)
[SUCCESS] Solved word EPITA correctly, got: Not found
[SUCCESS] Correctly solved all horizontal words

--- Vertical Tests ---
[SUCCESS] Solved word vertical correctly, got: (9, 7)(9, 0)
[SUCCESS] Solved word VERTICAL correctly, got: (9, 7)(9, 0)
[SUCCESS] Solved word LACITREV correctly, got: (9, 0)(9, 7)
[SUCCESS] Solved word khld correctly, got: (1, 8)(1, 5)
[SUCCESS] Solved word vetcal correctly, got: Not found
[SUCCESS] Correctly solved all vertical words

--- Diagonal Tests ---
[SUCCESS] Solved word diagonal correctly, got: (0, 1)(7, 8)
[SUCCESS] Solved word DIAGONAL correctly, got: (0, 1)(7, 8)
[SUCCESS] Solved word find correctly, got: (4, 8)(1, 5)
[SUCCESS] Solved word GOLDORAK correctly, got: (8, 1)(1, 8)
[SUCCESS] Solved word diagal correctly, got: Not found
[SUCCESS] Correctly solved all diagonal words

```

Figure 14: Exemple de foctionnement de l'algorithme de résolution sur une grille d'exemple avec des tests

2.4.2 Défis rencontrés

Lors de la réalisation de cet algorithme, plus précisément lors du passage d'un fichier texte à un tableau de caractères, une imprécision a ralenti le développement. En effet, Windows ou Linux n'enregistre pas les fins de ligne de la même façon, Windows met un `\n\r` à la fin et Linux met seulement un `\n`. Cela rendait l'algorithme non fonctionnel pour certains membres du groupe, mais pas pour tout le monde. Mais cette erreur mineure a été rapidement trouvée et réglée. Pour ce qui est du parcours dans toutes les directions, le code est très long, car il se répète huit fois pour les huit directions différentes, c'est pourquoi il est envisageable de modifier l'architecture de cette partie pour la rendre plus concise. Un autre problème a été détecté après la 1ère soutenance, lors de la création d'une batterie de tests pour le solver. En effet, une petite partie des mots n'étaient pas trouvés dans la grille, le code a donc été fix et aussi beaucoup raccourci par la même occasion car une partie de celui-ci se répétait beaucoup.

Quelques remarques

L'algorithme de résolution n'est pas parfait, sa complexité n'est pas la meilleure et il est possible d'améliorer son temps d'exécution. Premièrement, il fait son parcours comme dans l'algorithme de détection de lettres. En effet, il parcourt les lettres les unes après les autres, et dès qu'il tombe sur la première lettre, il visite ses voisins. Et ce pour chaque mot. Nous pourrions en effet imaginer qu'il ne parcourt qu'une seule fois la grille et qu'il renseigne les coordonnées de chaque lettre dans des listes triées, en effet, tout les "A" seraient ensembles, tout les "B" aussi, etc...

Cela permettrait que, lorsqu'on cherche un mot, on parcourt la liste des lettres égales à la première lettre du mot, on obtienne ses coordonnées dans la grille, et

ensuite on fait pareil pour la deuxième lettre, si ses coordonnées coïncident avec la lettre précédente, on continue le parcours, et ce jusqu'à être bloqué ou trouver le mot. Cela permettrait de faire diminuer largement le nombre de comparaisons car dans cet algorithme-ci, le programme ne comparera pas chaque lettre de la grille avant de trouver la bonne lettre, car il n'ira qu'à piocher dans la liste de la lettre qu'il recherche.

Cette amélioration demanderait donc de repenser toute la logique derrière l'algorithme du solver, et de plus, il est fort probable qu'un autre algorithme, encore plus optimisé soit applicable. Pour ce qui est de la clarté, lors de la réalisation du code du solver actuel, nous nous sommes toujours forcés de rendre le code clair en apportant le plus possibles des simplifications et ajouts pour le rendre le plus lisible possible.

2.5 Résolution de la grille

Une fois la détection de la grille effectuée, nous disposons des coordonnées de chaque lettre dans la grille et celles de chaque lettre dans la liste de mots. Grâce à ces coordonnées, nous pouvons appliquer un crop à chaque lettre et envoyer les images découpées au réseau de neurones, afin d'effectuer la reconnaissance des caractères.

2.5.1 Reconstruction de la grille

La première étape de la résolution de la grille consiste en la reconstruction de la grille sous forme de tableau à 2 dimensions de caractères. Pour ce faire, nous utilisons la structure `PipelineResult` de la pipeline, mais aussi une autre structure, `Letter`, qui stocke les coordonnées `x1`, `y1`, `x2`, `y2` de chaque lettre.

Nous commençons donc par la construction du tableau de `Letter` à partir d'une liste à une dimension de `Letter` dans la fonction `build_grid_from_image`. Nous trions les lettres en fonction de leur composante `y` puis, dans chaque ligne, nous trions les lettres en fonction de leur composante `x`. Cela permet d'avoir une grille avec toutes les lettres de la grille à la bonne place, car la pipeline ne prend pas forcément les lettres dans l'ordre.

Enfin, nous construisons le tableau de caractères à 2 dimensions, c'est-à-dire la liste de mots, dans la fonction `build_grid_array`. On récupère donc chaque lettre depuis la fonction précédente, et on prédit la lettre correspondante à la lettre découpée grâce à la fonction `predict_letter` du réseau de neurones. Cela permet de remplir le tableau de caractères avec les lettres prédites par le réseau de neurones, et d'obtenir à la fin la grille complète.

```

[INFO] Grille détectée : 17 lignes, 17 colonnes
[P X U T S I N I U P R V G B M D D]
[E H A A S P O J P E T B E Q Z L C]
[A U N T E G Q T L H R Z F A T O P]
[S H X F N G U A X E A A Y P O M H]
[Y O Y Y L D X L A K Y U Z L B S K]
[J X M U U G Q T R I M A G I N E B]
[H F N W F X H D P B B B T N V S K]
[H I I H D E S Q F U M Y E R N S X]
[R P B Z N H S D S L H O N B S S S]
[E H X A I Z I H A H O E S Q F E F]
[C W Z I M V D C J V S S I M G R W]
[L A I I R Z Q Q H X D Z O Z Q T R]
[W C A X E Z R G H A I Z N E C S E]
[B R H F O T G N I T S E R E O V Z]
[M W V W Q D U I H W Q T S B I M L]
[T D T O N Z C X X R G E L X H F Q]
[Q N E K S V M O T F A L A A E W B]

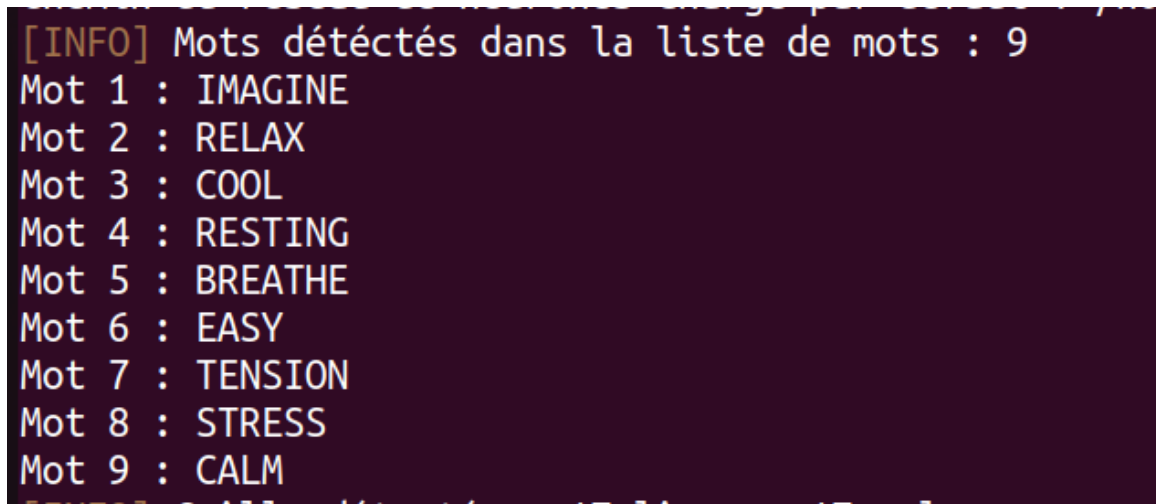
```

Figure 15: Reconstruction de la grille de l'image 1 du niveau 1

2.5.2 Reconstruction de la liste de mots

La reconstruction de la liste de mots est assez similaire à celle de la grille. En effet, on procède de la même manière pour obtenir en premier lieu la liste des mots à une dimension dans `build_words_list_from_image`. Nous faisons donc le tri des lettres de la liste de mots, exactement de la même manière que pour la grille, ce qui renvoie un tableau à 2 dimensions de `Letter`.

Ensuite, nous reconstruisons la liste de mots dans `build_words_list`, qui va découper chaque lettre de la liste précédente et l'envoyer au réseau de neurones qui va effectuer la reconnaissance de chaque caractère à partir des images découpées. On obtient donc la liste de mots contenant tous les mots qui contiennent toutes les lettres contenues dans chaque mot de la liste de mots de l'image.

A screenshot of a terminal window with a dark purple background. The text is displayed in a monospaced font, with the first line in yellow and the subsequent lines in white. The text lists nine words detected in a word list, numbered 1 through 9.

```
[INFO] Mots détectés dans la liste de mots : 9
Mot 1 : IMAGINE
Mot 2 : RELAX
Mot 3 : COOL
Mot 4 : RESTING
Mot 5 : BREATHE
Mot 6 : EASY
Mot 7 : TENSION
Mot 8 : STRESS
Mot 9 : CALM
```

Figure 16: Reconstruction de la liste de mots de l'image 1 du niveau 1

2.5.3 Résolution de la grille

Une fois les reconstructions de la grille et de la liste de mots effectuées, nous pouvons donc tout envoyer au programme du solver ce qui va résoudre chaque mot. On appelle donc la fonction `solve` qui renvoie les coordonnées de chaque mot dans la grille en 4 coordonnées $(x1, y1)(x2, y2)$.

Maintenant que l'on a les coordonnées de chaque mot dans la grille, on peut convertir ses coordonnées dans les coordonnées de l'image grâce aux coordonnées de la grille. C'est la fonction `get_solved_words_image_coos_drawing` qui s'occupe de cette conversion, et renvoie 8 entiers $(x1, y1), (x2, y2), (x3, y3), (x4, y4)$.

Cela permet donc de dessiner tous ces mots dans l'interface graphique, et de résoudre la grille complètement, en dessinant les rectangles autour des mots.

Pour dessiner ces rectangles, on dessine donc 4 lignes : une allant de $(x1, y1)$ à $(x2, y2)$, une allant de $(x2, y2)$ à $(x3, y3)$, puis de $(x3, y3)$ à $(x4, y4)$, et enfin de $(x4, y4)$ à $(x1, y1)$ pour fermer le rectangle.

Nous disposons donc d'une fonction `draw_line` qui utilise l'algorithme de Bresenham pour dessiner une ligne optimale reliant 2 points, ce qui fonctionne pour les lignes en diagonale dans le cas où le mot résolu est en diagonale.

Le dessin des lignes nécessite une dernière fonction, `draw_pixel`, qui dessine directement le pixel avec une couleur donnée sur l'image en coordonnées (x, y) .

Ces fonctions de dessins seront utilisés dans l'interface graphique lors de la résolution de la grille.

2.5.4 Défis rencontrés

La principale difficulté de la résolution de la grille vient du tri des lettres en différentes lignes et colonnes pour reconstruire les tableaux de caractères. Nous avons également eu des difficultés lors de l'appel au programme `solver` qui donne les coordonnées du mot dans la grille, étant donné que celui-ci prend en argument un tableau de caractères, et non un pointeur de pointeur de caractères comme l'on a sur nos grilles. Nous avons donc du changer cet argument en pointeur dynamique.

Enfin, la conversion des coordonnées des mots sur la grille en coordonnées des mots sur l'image est délicate car en fonction de l'orientation du mot (inversé ou non), il faut faire attention à toujours garder un rectangle pour dessiner.

2.6 Interface graphique

2.6.1 Bilan des réalisations

Interface de démonstration (Première soutenance)

L'interface graphique de démonstration, réalisée pour la première soutenance, permet au projet de prendre forme en permettant de voir le résultat de nos fonctions. Elle permet donc de tester le bon fonctionnement du projet en visualisant le résultat de la transformation d'une image par une fonction de traitement d'image, comme les conversions en niveaux de gris, ou en noir et blanc. Nous avons donc décidé d'avancer sur cette interface graphique pour la première soutenance, ce qui nous a permis de vérifier que le traitement d'image est fonctionnel.

L'interface graphique se lance avec une fenêtre *GtkWindow*, connectée à une application *GtkApplication*. La première version permettait de charger une image, et d'y appliquer diverses opérations de traitement d'image:

- Conversion en niveaux de gris
- Conversion en noir et blanc
- Rotation de l'image

En plus de ces trois opérations de traitement d'image, l'interface graphique permet également de:

- Réinitialiser l'image (efface toutes les transformations appliquées à l'image)
- Sauvegarder l'image vers un fichier PNG
- Fermer la fenêtre

Elle se lance avec `./build/ui <optionnel: chemin de l'image>`, et charge l'image donnée en paramètre, ou l'image `level_1_image_1.png` par défaut. L'utilisateur peut donc cliquer sur les boutons pour effectuer les transformations et voir les changements directement sur la fenêtre.

Enfin, les boutons appellent les fonctions respectives (par exemple, la conversion en niveau de gris appelle `convert_to_grayscale()`) et appellent ensuite les fonctions `update_image()` et `apply_transformations()` pour mettre à jour l'image affichée dans la fenêtre. Les boutons indiquent à l'utilisateur le traitement effectué, comme par exemple la sauvegarde de l'image avec le nom du fichier de l'image sauvegardée, pour pouvoir y accéder par la suite.

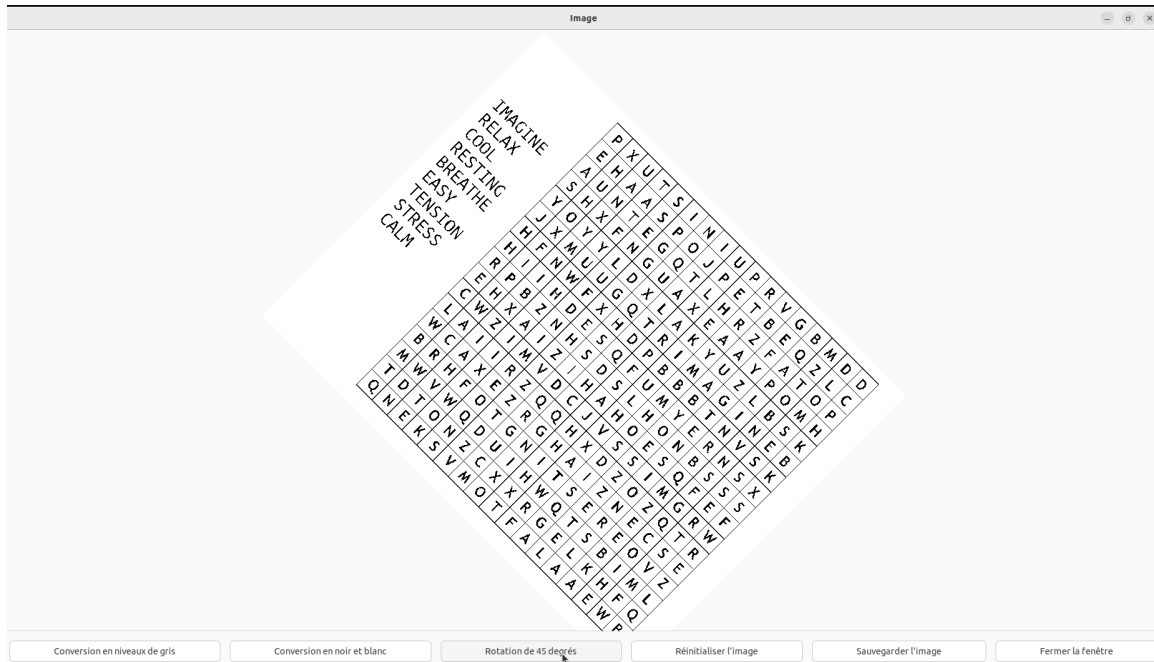


Figure 17: Interface graphique avec chargement de l'image *level_1_image_1.png*, conversion en noir et blanc puis rotation de 45 degrés vers la droite

Interface finale

L'interface graphique finale possède plusieurs sections importantes qui sont au nombre de trois. La première est le dessus de l'interface, à savoir le titre ainsi que le bouton quitter. Cliquer sur ce dernier permet à l'utilisateur de fermer totalement la fenêtre et ainsi quitter l'application de résolution.

La deuxième section importante est la partie centrale de l'image. Sur le côté gauche de cette partie centrale est affichée l'image que en cours de traitement ainsi que les différentes modifications apportées. Le côté droit quand à lui possède les boutons permettant l'accès au réseau de neurones. Le premier bouton permet de charger l'image dont la résolution est voulue. Cliquer dessus permettra à l'utilisateur de chercher l'image qu'il veut charger parmi les fichiers présents dans son ordinateur, il n'aura donc pas à mettre l'image dans un endroit précis, cela permet une meilleure liberté et moins de contraintes pour le choix de l'image.

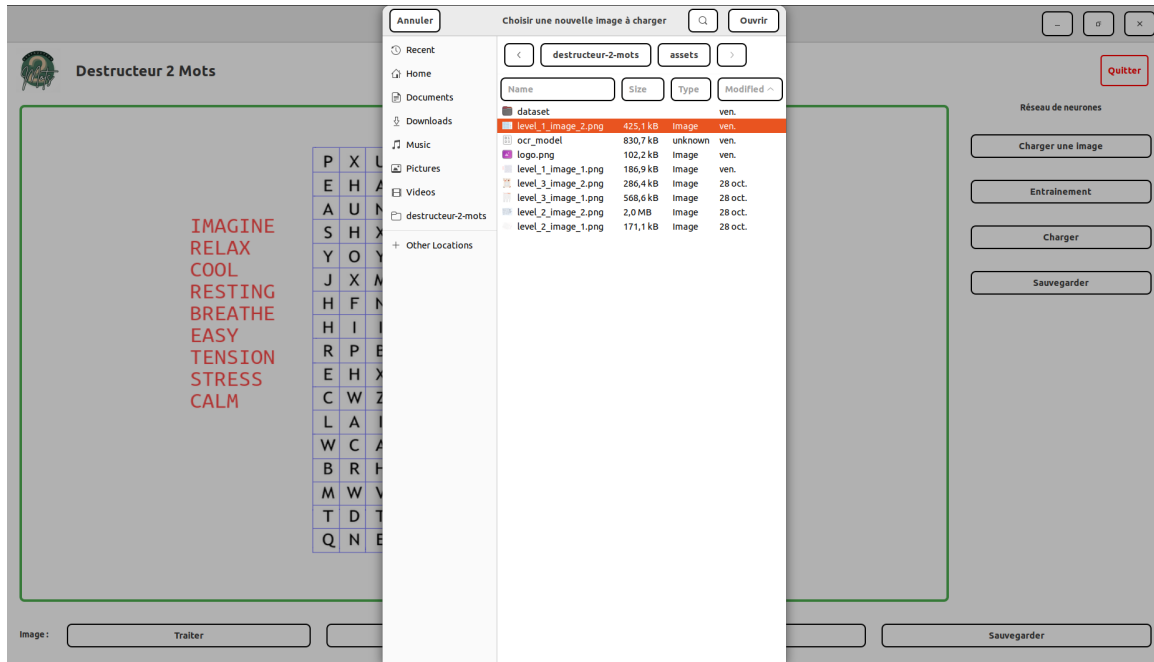


Figure 18: Fenêtre de chargement d'une nouvelle image après avoir cliqué sur le bouton "Charger"

Le bouton suivant permet de charger un dataset pour entraîner le réseau de neurones, il n'est pas nécessaire d'avoir créé un réseau au préalable car si il n'existe aucun réseau alors la fonction derrière s'occupera de créer le réseau en question puis l'entraînera avec le dossier dataset choisi. Dataset qui pourra se trouver où l'utilisateur veut car celui-ci peut choisir dans ses fichiers pour le donner.

Le bouton d'après permet de charger un réseau de neurones déjà existant, permettant ainsi de ne pas avoir à refaire tout l'entraînement d'un réseau. Enfin le dernier bouton permet justement de sauvegarder le réseau de neurones actuel, afin d'éviter de perdre tout l'entraînement réalisé. Enfin, l'utilisateur pourra choisir où enregistrer cet entraînement lorsqu'il cliquera sur le bouton sauvegarder.

La troisième et dernière section est l'ensemble des boutons en bas de l'interface qui permettent de gérer tout ce qui touche à la modification de l'image en cours de résolution. Le premier bouton permet de visualiser le prétraitement de l'image, à savoir la *binarisation*, la rotation automatique et le reste des traitements appliqués à l'image avant qu'elle soit envoyée au réseau de neurones. Il est important de savoir qu'on ne peut pas traiter deux fois de suite une même image car sinon celle-ci deviendrait blanche, perdant ainsi toute utilité, c'est pour cela que si l'utilisateur clique une deuxième fois sur ce bouton par inadvertance une fenêtre apparaîtra pour lui annoncer que l'image est déjà traitée et donc n'a plus besoin de l'être. Le deuxième bouton est celui qui permet d'appeler la fonction **solver** et donc de résoudre la grille, faisant ainsi apparaître la solution sur l'image en entourant les mots à trouver.

Ce bouton va dessiner tous les rectangles correspondants aux coordonnées des mots résolus dans l'image.

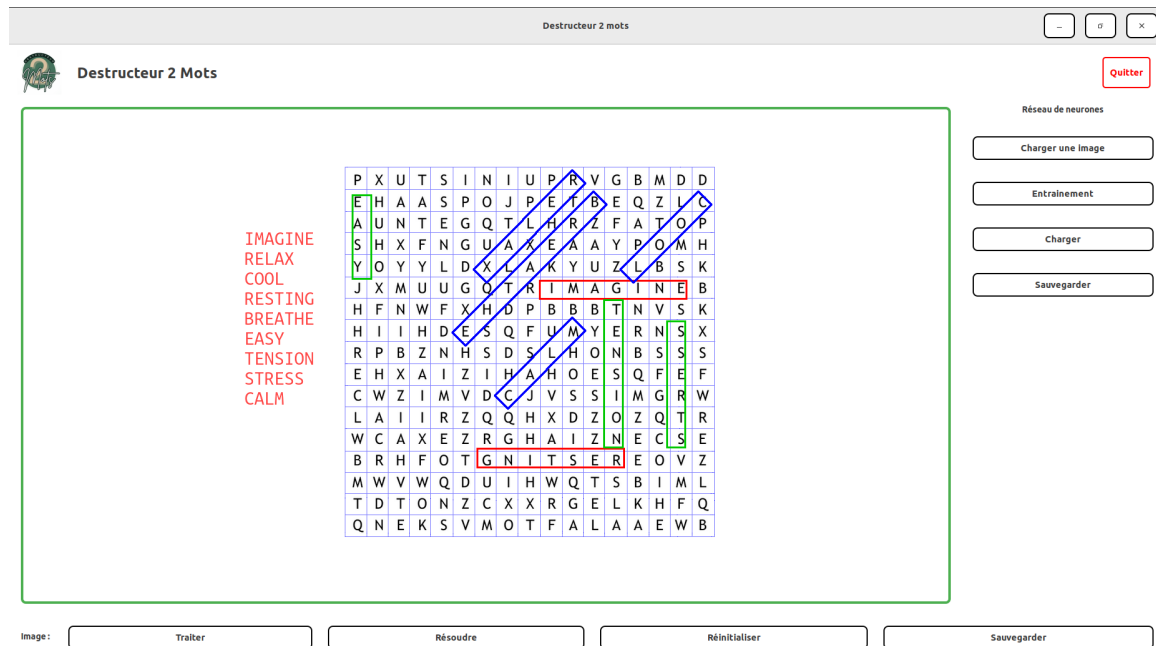


Figure 19: Résolution de la grille pour l'image 1 du niveau 1

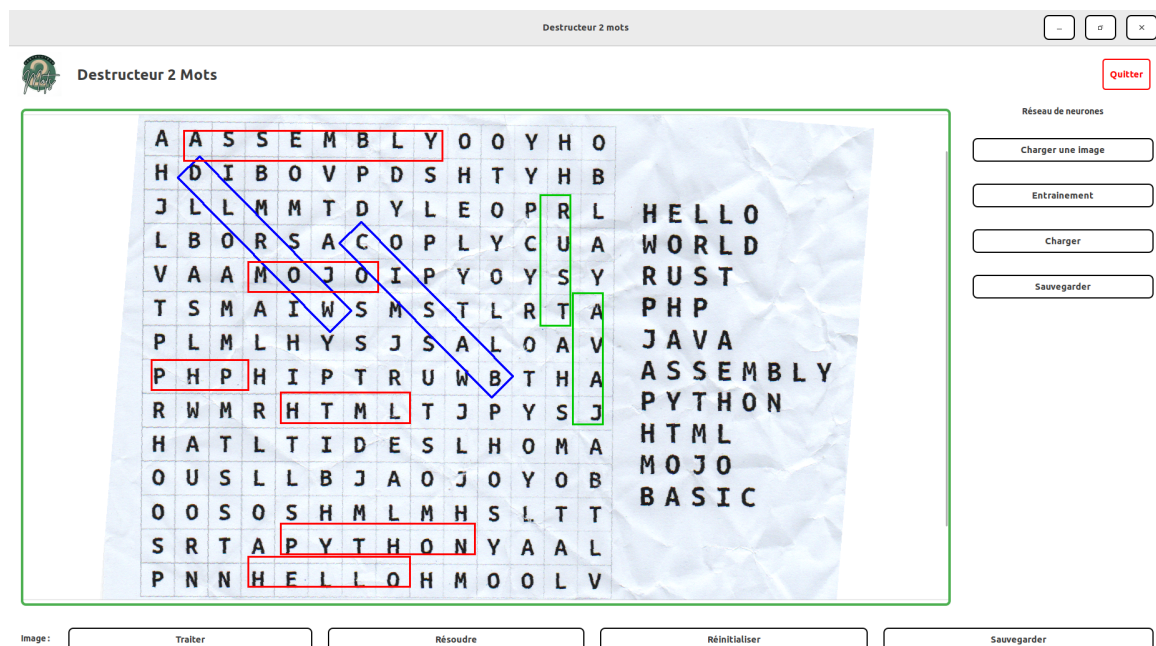


Figure 20: Résolution de la grille pour l'image 2 du niveau 2

Le troisième bouton permet de réinitialiser l'image en cas de mauvaises manipulations ou juste par envie de revenir à l'image de départ. Enfin, le dernier bouton

permet de sauvegarder l'image, encore une fois là où l'utilisateur le souhaite, il pourra choisir le chemin vers où enregistrer son image après avoir cliqué sur ce bouton.

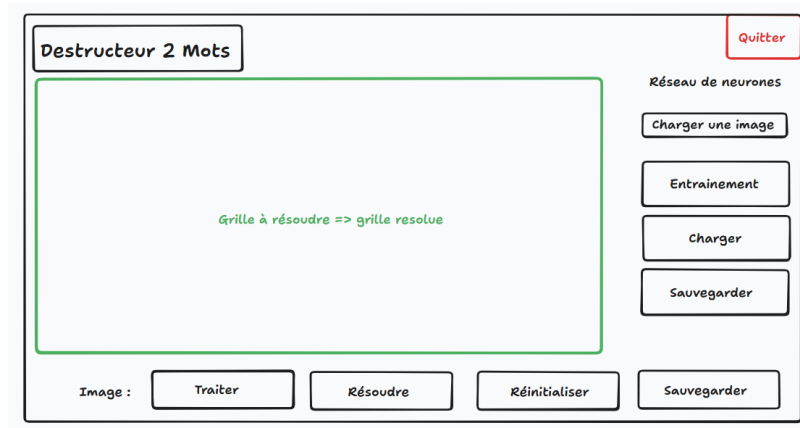


Figure 21: PoC de l'interface graphique de notre application

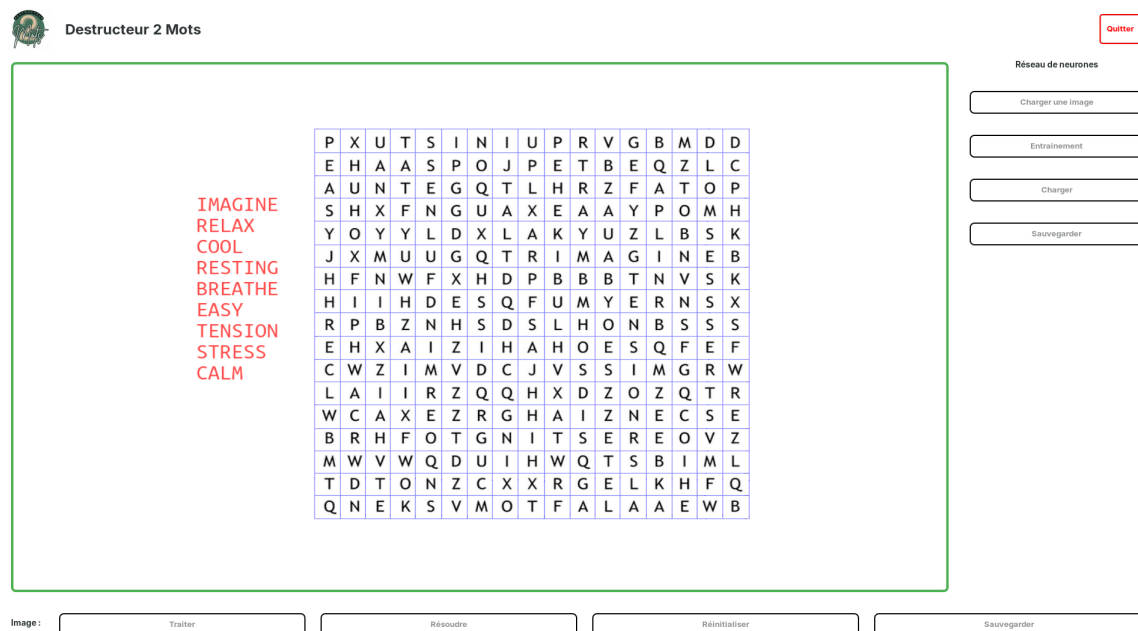


Figure 22: Interface graphique finale

Conclusion

Voici les fonctionnalités présentes dans l'interface graphique :

Pour tout ce qui concerne l'image, nous avons implémenté les fonctionnalités suivantes :

- Prétraitement de l'image
- Résolution de la grille
- Réinitialisation de l'image
- Sauvegarde de l'image en fichier png à l'endroit choisi par l'utilisateur

La partie droite de l'interface graphique est quant-à-elle axée sur la partie réseau de neurones, on retrouvera :

- De quoi charger l'image que l'on souhaite résoudre, l'utilisateur pourra choisir le chemin de l'image
- De quoi charger un dataset pour entraîner le réseau de neurones
- Charger un réseau préexistant
- Sauvegarde du réseau actuel en tant que fichier texte à l'endroit choisi par l'utilisateur

2.6.2 Défis rencontrés

La complexité du développement de l'interface graphique réside en la gestion des *pointeurs* pour chaque bouton, mais aussi la gestion des pointeurs pour les conteneurs. En effet, il vaut veiller à ce que chaque bouton soit connecté à la bonne fonction et qu'il soit empilé dans le bon ordre dans les boîtes.

En effet la complexité de l'interface graphique réside dans sa structure non visible, à savoir différents empilements de boîtes dans différents ordres. On retrouvera en effet 3 "couches" de niveau. La première couche permet d'aligner les 3 grandes parties de l'interface, à savoir la partie header, la partie centrale contenant l'image et le réseau, et la partie traitement de l'image.

La deuxième couche se trouve justement dans ces parties alignées, elle permet d'aligner les boutons de traitement d'image entre eux et d'allouer la place nécessaire pour afficher l'image et les boutons. La troisième et dernière couche permet d'aligner les différents boutons touchant au réseau de neurones. Cela implique donc une dépendance entre partie ce qui ne rend pas la tâche facile.

Une autre problématique qui s'est longtemps posée a été le problème du chemin du fichier pour l'image dont l'utilisateur souhaitait s'occuper. En effet différents problèmes d'allocations et d'accès ont été présents dans les différentes fonctions qui traitaient l'image.

De plus, comme l'utilisateur doit pouvoir effectuer plusieurs traitements à la fois sur une même image, nous avons créé une structure *AppData* qui contient des pointeurs sur les *Pixbufs* des images originales et transformées.

Axes d'améliorations possibles :

- Meilleur visuel pour la fenêtre graphique, différentes couleurs etc...
- Possibilité de résoudre soit même la grille pour entrainer le réseau de neurones
- Possibilité pour l'utilisateur de faire apparaître la solution étape par étape, en cliquant sur un bouton "Suivant"
- Possibilité d'avoir 2 images en même temps et comparer 2 neurones différents

3 Gestion de projet

3.1 Répartition et avancement du travail

Domaines de travail	Répartition				Avancement	
Soutenances	Kristen	Romain	Liam	Timothée	Novembre	Décembre
Réseau de neurones	R				30%	100%
Structure et conception	R				30%	100%
Reconnaissance des lettres	R				0%	100%
Sauvegarde et chargement de l'entraînement	R				0%	100%
Traitement de l'image			R	S	90%	100%
Chargement et affichage d'une image			R	S	100%	100%
Suppression des couleurs			R	S	100%	100%
Rotation			R	S	80%	100%
Détection de grille/lettres/mots		R			90%	100%
Détection de la position grille		R			90%	100%
Détection de la position liste de mots		R			90%	100%
Détection des cases et lettres		R			90%	100%
Résolution de la grille		S		R	50%	100%
Reconstruction de la grille				R	0%	100%
Reconstruction de la liste de mots				R	0%	100%
Résolution de la grille		S		R	80%	100%
Interface graphique	S		R	S	10%	95%

Table 2: R correspond au rôle de Responsable et S correspond au rôle de Suppléant. Les sous-catégories détaillent les étapes du projet.

4 Conclusion

Lors de la réalisation de ce projet, nous avons pu créer, pas à pas, les différentes blocs le composant.

Nous nous sommes renseignés sur les différentes façon d'arriver à notre objectif final et nous avons beaucoup appris sur les réseaux de neurones, traitement d'image ou encore interface graphique.

Au début de l'année, nous pensions que le réseau de neurone était la partie la plus difficile, longue et intense du projet, mais finalement, chaque tâche se valait, rendant chaque membre du groupe utile au projet.

De plus, ce projet nous a aussi appris le travail de groupe, que ça soit dans la programmation en rendant notre code le plus clair possible, mais aussi dans la gestion d'un répertoire Github.

Enfin, nous sommes satisfaits du résultat final, le gros du projet à en effet été réalisé, même si nous pourrions tout de même l'améliorer en le renforçant dans sa détection de caractères, ou encore dans l'entraînement du réseau de neurone sur un plus gros dataset. Cela nous permettrait sans doute de résoudre les images du niveau 3.

5 Annexes

5.0.1 Termes techniques et définitions

PoC : *Proof of Concept*, désigne une version préliminaire d'un projet permettant de valider la faisabilité technique d'une idée avant son développement complet.

OCR : *Optical Character Recognition*, technologie permettant de reconnaître et de convertir du texte présent dans une image en texte éditable.

GTK : *GIMP Toolkit*, bibliothèque utilisée pour créer des interfaces graphiques (GUI) en C, compatible avec Linux, Windows et macOS.

GDK : *GIMP Drawing Kit*, couche intermédiaire entre GTK et le système de fenêtrage, utilisée pour la gestion des événements et le rendu graphique de bas niveau.

XOR/AND/OR/XNOR : Opérateurs logiques utilisés dans les circuits et réseaux de neurones pour combiner des valeurs binaires selon des règles booléennes.

Rétropropagation : Algorithme d'apprentissage supervisé qui ajuste les poids d'un réseau de neurones en fonction de l'erreur calculée entre la sortie attendue et la sortie réelle.

Descente de gradient : Méthode d'optimisation consistant à modifier progressivement les poids d'un modèle afin de minimiser une fonction de coût.

Perceptron : Neurone artificiel simple qui calcule une somme pondérée de ses entrées, applique une fonction d'activation et produit une sortie binaire.

Fonction d'activation : Fonction mathématique appliquée à la sortie d'un neurone pour introduire de la non-linéarité et permettre l'apprentissage de relations complexes.

GdkPixbuf : Structure de données de la bibliothèque GDK utilisée pour représenter et manipuler des images en mémoire (chargement, affichage, sauvegarde).

PNG : *Portable Network Graphics*, format d'image sans perte qui prend en charge la transparence et est couramment utilisé pour les interfaces et le web.

Filtre de Sobel : Opération de détection de contours qui calcule le gradient d'intensité d'une image afin de mettre en évidence les zones de transition rapide de luminosité.

EMNIST/MNIST : *(Extended) Modified National Institute of Standards and Technology database*, bases de données de référence contenant des caractères manuscrits (chiffres pour MNIST, extension aux lettres pour EMNIST), largement utilisées pour l'entraînement et l'évaluation des algorithmes de classification d'images.

Softmax : *Fonction d'activation*, transforme un vecteur de valeurs réelles (logits) en une distribution de probabilités dont la somme vaut 1, généralement utilisée dans la couche de sortie pour la classification multi-classes.

Cross Entropy : *Entropie croisée*, fonction de coût utilisée pour les problèmes de classification, qui mesure la divergence entre les probabilités prédites par le modèle et les étiquettes réelles (souvent couplée avec Softmax).

Erreur Quadratique : *Mean Squared Error (MSE)*, fonction de perte calculant la moyenne des carrés des écarts entre les valeurs prédites et les valeurs cibles, typiquement privilégiée pour les problèmes de régression.

ReLU : *Rectified Linear Unit*, fonction d'activation qui introduit de la non-linéarité en conservant les valeurs positives et en annulant les négatives ($f(x)=\max(0,x)$), favorisant un apprentissage rapide et réduisant le problème de disparition du gradient.

Couche Cachée : *Hidden Layer*, strate intermédiaire d'un réseau de neurones située entre l'entrée et la sortie, où s'effectuent les calculs pondérés et les transformations non linéaires permettant au modèle d'extraire des caractéristiques complexes.

Sigmoïde : *Sigmoid Function*, fonction d'activation en forme de S définie par $f(x) = \frac{1}{1+e^{-x}}$, qui comprime toute valeur réelle dans l'intervalle $]0,1[$, souvent utilisée pour la classification binaire.

5.0.2 Bibliographie

- *Neural Networks And Deep Learning* : <http://neuralnetworksanddeeplearning.com/>
- *Sausheong's article* : <https://sausheong.github.io/posts/how-to-build-a-simple-artifi>
- *Mots mêlés* : https://en.wikipedia.org/wiki/Word_search
- *Détection des lettres/mots/grilles* : https://en.wikipedia.org/wiki/Line_detection
- *Documentation GTK3* : <https://docs.gtk.org/gtk3/>
- *nicolas.thiery.name* : <https://nicolas.thiery.name/Enseignement/Info111/Projet-Image/partie-3.html>